

DepChain: A Simplified Permissioned Blockchain System with High Dependability Guarantees

Francisco Fonseca^[102492], Gonalo Rua^[102604], and Joao Gouveia^[102611]

Instituto Superior Tecnico, Universidade de Lisboa, Portugal

Abstract. DepChain is a simplified permissioned blockchain system designed with a strong focus on dependability and Byzantine fault tolerance. This project was developed in two stages as part of the Highly Dependable Systems course. In the first stage, we implemented a Byzantine consensus algorithm based on the Read/Write Epoch Consensus model, enhanced with layered communication abstractions such as Fair Loss Links, Stubborn Links, and Authenticated Perfect Links. Our implementation ensures strong safety and liveness guarantees in the presence of up to f Byzantine members in a system of $n \geq 3f + 1$ nodes. In the second stage, we built a flexible blockchain layer capable of processing cryptocurrency transfers and executing Solidity smart contracts within a sandboxed EVM environment using Hyperledger Besu. Our system supports generic contract execution, including dynamic storage inspection and return value propagation to clients. Security measures include signed communication, replay protection via nonces, and contract-level access control through a blacklist. The system is tested through unit and integration tests, and a TUI frontend is provided for demonstration. DepChain provides a strong foundation for building highly dependable decentralized systems with support for extensible application logic.

Keywords: Blockchain · Byzantine Consensus · Smart Contracts · Hyperledger Besu · Dependability.

1 Introduction

Distributed systems that require a high degree of reliability often rely on consensus protocols that tolerate arbitrary (Byzantine) failures. Blockchain systems are a prominent example where security, integrity, and availability must be preserved even under the presence of malicious actors. The goal of this project is to design and implement **DepChain**, a permissioned blockchain system with high dependability guarantees, developed as part of the Highly Dependable Systems course.

DepChain was built in two incremental stages. The first stage focused on implementing a consensus layer based on the Byzantine Read/Write Epoch Consensus algorithm. To support this, we implemented a layered communication stack over UDP, including abstractions such as FairLossLinks, StubbornLinks (with

signed acknowledgments to prevent denial-of-service attacks), ConditionalCollect, and AuthenticatedPerfectLinks. Our consensus engine supports deciding a sequence of values in a single execution, inspired by the Multi-Paxos model.

In the second stage, we developed a blockchain execution engine capable of processing cryptocurrency transactions and executing Solidity-based smart contracts. We used Hyperledger Besu as an embedded EVM to provide deterministic contract execution and storage inspection. The system supports generic contracts beyond those required by the project, and returns their results to the clients. A client library and a terminal-based UI facilitate interaction and testing.

In the remainder of this report, we describe the architecture, security considerations, and implementation choices.

2 System Architecture

The DepChain system consists of three major components: (1) the *client library*, (2) the *blockchain members*, and (3) the *consensus protocol stack*. These components collaborate to provide a robust blockchain service with Byzantine fault tolerance and support for generic smart contract execution.

2.1 Client Library and Frontend

The client library exposes a simplified interface for interacting with the blockchain. Clients can submit requests of three types: `CHECK_BALANCE`, `READ_STATE`, and `TRANSACTION`. Requests are serialized using Protocol Buffers and sent over TCP to the leader of the blockchain. Each request is signed using the client’s private key, and the leader validates the signature and source address before further processing. Nonces are used for `TRANSACTION` requests to prevent replay attacks, but are not enforced for read-only operations.

For demonstration and testing, we developed a text-based user interface (TUI) that connects to the client library and enables users to perform various operations, including DepCoin transfers, IST Coin operations, and custom smart contract execution.

Responses are structured to return requested data (e.g., balances or storage values) or transaction results (e.g., return values from smart contracts). Each transaction’s result is saved as part of the block and returned to the client upon execution.

2.2 Blockchain Members and State Management

Blockchain members are responsible for maintaining the replicated state of the blockchain and executing transactions. Each member runs a local instance of the consensus protocol and executes decided blocks to update the blockchain state.

The blockchain state is stored on disk in JSON format, starting from a genesis block that initializes accounts, balances, and deployed contracts. The state

includes both Externally Owned Accounts (EOA) and Contract Accounts, following the Ethereum account model. Each account has an address, balance, and, in the case of contracts, bytecode and persistent storage.

Transactions are grouped into blocks and decided by the consensus protocol. Upon reaching consensus, the decided block is executed sequentially by each blockchain member, ensuring deterministic updates to the global state.

Block Proposal Trigger. The leader is responsible for proposing new blocks for consensus. A new block is proposed under one of the following conditions:

1. The block reaches its maximum capacity (i.e., a predefined number of transactions).
2. A timer expires and there are pending transactions waiting to be proposed.

If the timer expires but no transactions are available, the leader will propose a block immediately upon receiving the next transaction. This ensures timely block production without requiring empty blocks.

2.3 Communication and Serialization

All messages between clients and blockchain members, as well as between the blockchain members themselves, are serialized using Protocol Buffers. Communication is performed using UDP sockets, with custom abstractions built on top to provide reliability and security. These abstractions include:

- **FairLossLinks (FLL)**: A basic unreliable communication abstraction that mimics the behavior of UDP.
- **StubbornLinks (SL)**: Ensures eventual delivery by repeatedly sending messages until an acknowledgment is received. To prevent denial-of-service attacks, acknowledgments are signed.
- **AuthenticatedPerfectLinks (APL)**: Builds on top of StubbornLinks to ensure that messages are delivered exactly once and are authenticated using digital signatures.
- **ConditionalCollect (CC)**: Used in the consensus protocol to collect messages from a quorum of peers under certain conditions.

2.4 Execution Engine

DepChain uses Hyperledger Besu as an embedded Ethereum Virtual Machine (EVM) for executing smart contracts. Contracts are compiled from Solidity and deployed at genesis. The execution environment includes:

- A custom **SimpleWorld** state provider to simulate the world state.
- Storage tracking using the EVM tracer output to identify which storage slots were accessed.
- Return value extraction to communicate smart contract responses back to the client.

This design allows DepChain to support the execution of arbitrary smart contracts beyond those required by the project (e.g., IST Coin and Blacklist).

3 Byzantine Fault-Tolerant Consensus

At the core of DepChain lies a Byzantine consensus protocol based on the Read/Write Epoch Consensus model studied in class. Our implementation follows the version described in Algorithms 5.17 and 5.18 of the course book, with key enhancements to support the repeated agreement on sequences of values, inspired by Multi-Paxos.

3.1 Protocol Overview

The system assumes a static membership with n blockchain members, where up to f may behave arbitrarily (i.e., exhibit Byzantine faults). Correctness is guaranteed as long as $n \geq 3f + 1$. We also assume the leader is correct for this stage, simplifying the implementation by removing the need to handle epoch changes.

The consensus algorithm consists of the following primitives:

- **Init**: Initializes the state for consensus execution.
- **Propose**: Invoked by the leader to propose a value (e.g., a list of transactions forming a block).
- **Decide**: Invoked by each member when a value is agreed upon.

In our implementation, consensus is triggered only by the leader. The leader collects client transactions, verifies their validity, and proposes them as a batch (a block) through the consensus algorithm. Once a decision is made, all members execute the block to update their local blockchain state.

3.2 Multi-Value Consensus Execution

Our implementation allows the leader to propose and decide on a sequence of values in the same consensus instance. This model is similar to Multi-Paxos and results in a single instance of consensus deciding all the blocks. This optimization preserves the original algorithm's safety properties and improves performance.

The **Decide** primitive, upon invocation, notifies the upper layer (i.e., the blockchain module) to apply the block and persist the new state. Each member independently executes the block and updates its local JSON-based ledger and world state.

3.3 Threat Model and Guarantees

Our protocol provides the following guarantees:

- **Safety**: No two correct nodes decide differently, even in the presence of up to f Byzantine nodes.
- **Liveness**: Progress is guaranteed when the leader is correct and the network behaves fairly.

- **Authenticity:** Messages are signed and verified using public keys, ensuring they are not forged or tampered with.

While our current implementation does not support leader replacement, we designed the communication and consensus layers with future extensions in mind. In particular, all messages carry enough metadata and authentication to enable safe epoch transitions if needed.

4 Blockchain Implementation

DepChain’s blockchain layer is responsible for managing account balances, executing transactions, maintaining contract state, and persisting the evolving world state. It is designed to be modular and extensible, supporting arbitrary smart contracts written in Solidity and compiled to EVM bytecode.

4.1 Accounts and Transactions

DepChain follows the Ethereum account model, distinguishing between:

- **Externally Owned Accounts (EOA):** Controlled by clients using public/private key pairs. These accounts hold balances and nonces and can initiate transactions.
- **Contract Accounts:** Contain EVM bytecode and a persistent key-value store. Transactions directed to these accounts trigger contract execution.

Transactions can be of type **TRANSFER** or smart contract **EXECUTION**. Each transaction includes fields such as sender/receiver addresses, amount, nonce, payload (e.g., contract calldata), and a digital signature. The leader verifies signatures and ensures message integrity before appending the transaction to a proposed block. Nonce validation is deferred until the execution phase to handle multiple transactions from the same sender within the same block.

4.2 Smart Contract Execution

Smart contract execution is powered by Hyperledger Besu’s EVM implementation, as we are using it as a "sandbox". Contracts are compiled from Solidity and loaded into the sandbox at transaction execution time.

A key feature of our implementation is the ability to track and persist only the modified storage keys after contract execution. This is done by capturing the output of Besu’s EVM tracer and extracting accessed storage slots using operations **SLOAD** and **SSTORE**.

The tracer output is also parsed to extract the return value of the contract call, which is forwarded back to the client.

4.3 Smart Contract Deployment

Although we did not implement a separate transaction type for contract deployment during runtime, we developed a flexible mechanism to deploy contracts programmatically. This mechanism is used during genesis block creation and is capable of:

- Executing the deployment bytecode using the EVM.
- Extracting the resulting runtime bytecode.
- Initializing contract storage from tracer output.

This setup allows the seamless deployment of contracts, making the future implementation of contract deployment transactions straightforward.

5 Security and Dependability Guarantees

DepChain is designed to operate correctly in the presence of Byzantine failures across both clients and servers. We reasoned carefully about the potential threats introduced by arbitrary behavior and implemented a variety of protection mechanisms across all layers of the system.

5.1 Threat Model

Our threat model includes:

- **Byzantine servers:** Any subset of up to f blockchain members can behave arbitrarily, including forging messages, dropping or reordering them, or colluding with other malicious actors.
- **Byzantine clients:** Clients may send malformed or invalid requests, replay old messages, or submit conflicting transactions.
- **Unreliable network:** Messages may be dropped, delayed, duplicated, or tampered with, and communication channels are insecure.

5.2 Integrity and Authenticity of Messages

All communication between clients and the blockchain (as well as between blockchain members) is cryptographically signed:

- Each message includes a digital signature generated using the sender’s private key.
- Public keys of all participants (clients and members) are pre-distributed and known to the system.
- Upon receipt, each message is verified using the corresponding public key. If verification fails, the message is immediately discarded.

This prevents unauthorized message injection and guarantees non-repudiation.

5.3 Denial-of-Service Mitigation

Stubborn Links require that acknowledgments are signed, which protects against a specific class of denial-of-service attacks. Without this, a malicious node could flood the system with fake ACKs and halt retransmission prematurely. By validating ACK signatures, we ensure that only legitimate acknowledgments influence the behavior of reliable links.

5.4 Transaction Validation Pipeline

Before including a transaction in a proposed block, the leader performs:

- **Signature verification:** Ensures the transaction was signed by the sender’s private key.
- **Address matching:** The sender’s address included in the transaction is re-computed by hashing the provided public key, following the same derivation scheme used in Ethereum (i.e., taking the last 20 bytes of the Keccak-256 hash of the public key). This derived address is then matched against the one declared in the transaction’s `from` field.
- **Structural integrity:** Validates required fields and ensures the payload is well-formed.

Only transactions that pass all validation steps are appended to the block.

5.5 Nonce-Based Replay Protection

Each EOA account in the system maintains a nonce, used to prevent replay attacks. As stated before, nonce validation is deferred until transaction execution (i.e., after the block is decided), allowing multiple transactions from the same client to be batched into a single block:

- If the nonce in a transaction is greater than the stored nonce, the transaction is accepted and the stored nonce is updated.
- If the nonce is stale (equal or smaller), the transaction is rejected.

5.6 Client Library Trust Model

The client library is assumed to be trusted by the application using it. This trust model excludes attacks where a compromised library attempts to mislead an honest client, as per the project guidelines.

5.7 Resilience to Byzantine Members

To ensure the correctness and robustness of DepChain, we conducted extensive testing across both the consensus and blockchain execution layers. Our evaluation includes unit tests and an interactive terminal-based client for live demonstrations. Further testing details, usage instructions, and walkthroughs are provided in the accompanying README file.

5.8 Data Integrity and Auditability

Each block in the chain includes a cryptographic hash of its contents and a reference to the previous block. This creates an immutable chain of blocks that can be audited and verified for tampering. State snapshots are stored on disk after every block execution, enabling rollback, verification, and recovery.

6 Testing and Evaluation

To ensure the correctness and robustness of DepChain, we conducted extensive testing across both the consensus and blockchain execution layers. Our evaluation includes unit tests, and an interactive terminal-based client for live demonstrations. This is better explained in the README file.

7 Conclusion and Future Work

In this project, we developed **DepChain**, a highly dependable permissioned blockchain system that integrates Byzantine consensus, layered secure communication, and a flexible smart contract execution environment.

Our system enforces strong security guarantees, including message authentication, replay protection, access control enforcement, and state integrity. It is resilient to a wide range of Byzantine behaviors from both clients and servers, and has been thoroughly tested through unit tests and an interactive demo application.

7.1 Future Work

While DepChain meets the project goals and offers strong dependability guarantees, there are several directions for future improvement. One natural extension would be to support dynamic membership and Byzantine leader replacement, which would enhance the system's ability to recover from faults and evolve over time. Another potential improvement is the addition of contract deployment through standard transactions, allowing users to publish new smart contracts post-genesis rather than relying on pre-defined deployments. These enhancements would make DepChain more flexible and closer to real-world blockchain platforms.

Acknowledgments. This work was developed as part of the course *Highly Dependable Systems (SEC-2425)* at Instituto Superior Técnico, under the guidance of Prof. Paolo Romano and Prof. Miguel Matos. We thank the teaching team for the well-designed labs and support materials, including the Hyperledger Besu boilerplate and consensus algorithm references.

References

1. Cachin, C., Guerraoui, R., Rodrigues, L.: Introduction to Reliable and Secure Distributed Programming. 2nd edn. Springer, Berlin (2011)
2. Hyperledger Besu - Ethereum Client. <https://github.com/hyperledger/besu>, last accessed 2025/03/30
3. Ethereum Foundation: Remix IDE - Web-based Solidity Development Environment. <https://remix.ethereum.org>, last accessed 2025/03/30
4. Ethereum Foundation: ERC-20 Token Standard. <https://ethereum.org/en/developers/docs/standards/tokens/erc-20>, last accessed 2025/03/30
5. OpenZeppelin - Secure Smart Contract Library. <https://github.com/OpenZeppelin/openzeppelin-contracts>, last accessed 2025/03/30
6. Springer: Information for LNCS Authors. <https://www.springer.com/gp/computer-science/lncs/conference-proceedings-guidelines>, last accessed 2025/03/30
7. Google: Protocol Buffers Documentation. <https://developers.google.com/protocol-buffers>, last accessed 2025/03/30